



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ	ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА	СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

Индивидуальное домашнее задание

«Разработка консольного приложения с применением ООП»

ДИСЦИПЛИНА: «Архитектура АСОИУ»

Выполнил: студент гр. ИУ5-23Б

_____ (Ицхакина В.П.)
(Подпись) (Ф.И.О.)

Проверил:

_____ (Гапанюк Ю. Е.)
(Подпись) (Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:
- Оценка:

2026 г.

Цель работы: изучить и реализовать консольное приложение на языке C# с применением принципов ООП для работы с базой данных SQLite, паттерна Fluent Interface для построения отчётов, а также операций CRUD для управления данными.

Задание: разработать консольное приложение для управления заказами магазинов (вариант 11, группа 3). Приложение должно хранить данные в SQLite, позволять добавлять, редактировать и удалять записи через консольное меню, формировать отчёты с помощью паттерна Fluent Interface, включая итоговую строку Footer (дополнительное задание группы 3В).

1. Листинг кода:

```
// ===== Manufacturer.cs =====  
using System;  
  
namespace ElectronicsStore  
{  
    public class Manufacturer  
    {  
  
        public int Id { get; set; }  
  
        public string Name { get; set; }  
  
        public Manufacturer(int id, string name)  
        {  
            Id = id;  
            Name = name;  
        }  
  
        public Manufacturer() : this(0, "") { }  
  
        public override string ToString() => $"[ID: {Id}] {Name}";  
    }  
}  
// ===== Smartphone.cs =====  
using System;
```

```

namespace ElectronicsStore
{

    public class Smartphone
    {

        public int Id { get; set; }

        public int ManufacturerId { get; set; }

        public string Name { get; set; }

        private int _price;

        public int Price
        {
            get => _price;
            set
            {
                if (value < 0)
                    throw new ArgumentException("Критическая ошибка: Стоимость
смартфона не может быть меньше нуля!");
                _price = value;
            }
        }

        public Smartphone(int id, int manufacturerId, string name, int price)
        {
            Id = id;
            ManufacturerId = manufacturerId;
            Name = name;
            Price = price;
        }

        public Smartphone() : this(0, 0, "", 0) { }

        public override string ToString() => $"[ID: {Id}] {Name} | Код бренда:
#{ManufacturerId} | Цена: {Price} руб.";
    }
}

```

```

    }
}
// ===== DatabaseManager.cs =====
using System;
using System.Collections.Generic;
using Microsoft.Data.Sqlite;

namespace ElectronicsStore
{
    public class DatabaseManager
    {
        private readonly string _connString = "Data Source=smartphones.db";

        public void Init()
        {
            using var conn = new SqliteConnection(_connString);
            conn.Open();
            var cmd = conn.CreateCommand();

            cmd.CommandText = @"
CREATE TABLE IF NOT EXISTS manufacturer (
    mfg_id INTEGER PRIMARY KEY,
    mfg_name TEXT NOT NULL
);
CREATE TABLE IF NOT EXISTS smartphone (
    phn_id INTEGER PRIMARY KEY AUTOINCREMENT,
    mfg_id INTEGER NOT NULL,
    phn_name TEXT NOT NULL,
    phn_price INTEGER NOT NULL
);";
            cmd.ExecuteNonQuery();

            cmd.CommandText = "SELECT COUNT(*) FROM manufacturer";
            if (Convert.ToInt32(cmd.ExecuteScalar()) == 0)
            {
                cmd.CommandText = @"
INSERT INTO manufacturer VALUES (1, 'Apple'), (2, 'Samsung'), (3,
'Xiaomi'), (4, 'Huawei');

INSERT INTO smartphone (mfg_id, phn_name, phn_price) VALUES
(1, 'iPhone 15 Pro', 120000), (1, 'iPhone 14', 85000), (1, 'iPhone
SE', 45000),

```

```

                (2, 'Galaxy S24 Ultra', 130000), (2, 'Galaxy A55', 38000), (2,
'Galaxy XCover 7', 42000),
                (3, 'Xiaomi 14 Ultra', 110000), (3, 'Redmi Note 13 Pro', 28000),
(3, 'Poco F6', 35000),
                (4, 'Pura 70 Pro', 90000), (4, 'Nova 12 SE', 27000), (4, 'Mate 60
Pro', 105000));";

                cmd.ExecuteNonQuery();
            }
        }

        public List<Manufacturer> GetBrands()
        {
            var list = new List<Manufacturer>();
            using var conn = new SqlConnection(_connString);
            conn.Open();
            var cmd = conn.CreateCommand();
            cmd.CommandText = "SELECT mfg_id, mfg_name FROM manufacturer ORDER BY
mfg_id";

            using var r = cmd.ExecuteReader();
            while (r.Read())
            {
                list.Add(new Manufacturer(r.GetInt32(0), r.GetString(1)));
            }

            return list;
        }

        public List<Smartphone> GetPhones()
        {
            var list = new List<Smartphone>();
            using var conn = new SqlConnection(_connString);
            conn.Open();
            var cmd = conn.CreateCommand();
            cmd.CommandText = "SELECT phn_id, mfg_id, phn_name, phn_price FROM
smartphone ORDER BY phn_id";

            using var r = cmd.ExecuteReader();
            while (r.Read())
            {
                list.Add(new Smartphone(r.GetInt32(0), r.GetInt32(1),
r.GetString(2), r.GetInt32(3)));
            }

            return list;
        }

        public Smartphone GetPhoneById(int id)
        {
            using var conn = new SqlConnection(_connString);

```

```

        conn.Open();
        var cmd = conn.CreateCommand();
        cmd.CommandText = "SELECT phn_id, mfg_id, phn_name, phn_price FROM
smartphone WHERE phn_id = @id";
        cmd.Parameters.AddWithValue("@id", id);
        using var r = cmd.ExecuteReader();
        if (r.Read())
            return new Smartphone(r.GetInt32(0), r.GetInt32(1),
r.GetString(2), r.GetInt32(3));
        return null;
    }

    public void AddPhone(int mfgId, string name, int price)
    {
        using var conn = new SqlConnection(_connString);
        conn.Open();
        var cmd = conn.CreateCommand();
        cmd.CommandText = "INSERT INTO smartphone (mfg_id, phn_name,
phn_price) VALUES (@mfg, @name, @price)";
        cmd.Parameters.AddWithValue("@mfg", mfgId);
        cmd.Parameters.AddWithValue("@name", name);
        cmd.Parameters.AddWithValue("@price", price);
        cmd.ExecuteNonQuery();
    }

    public void UpdatePhone(Smartphone phn)
    {
        using var conn = new SqlConnection(_connString);
        conn.Open();
        var cmd = conn.CreateCommand();
        cmd.CommandText = "UPDATE smartphone SET mfg_id = @mfgId, phn_name =
@name, phn_price = @price WHERE phn_id = @id";
        cmd.Parameters.AddWithValue("@id", phn.Id);
        cmd.Parameters.AddWithValue("@mfgId", phn.ManufacturerId);
        cmd.Parameters.AddWithValue("@name", phn.Name);
        cmd.Parameters.AddWithValue("@price", phn.Price);
        cmd.ExecuteNonQuery();
    }

    public void DeletePhone(int id)
    {
        using var conn = new SqlConnection(_connString);

```

```

        conn.Open();
        var cmd = conn.CreateCommand();
        cmd.CommandText = "DELETE FROM smartphone WHERE phn_id = @id";
        cmd.Parameters.AddWithValue("@id", id);
        cmd.ExecuteNonQuery();
    }

    public (string[] cols, List<string[]> rows) RunSql(string sql)
    {
        using var conn = new SqlConnection(_connString);
        conn.Open();
        var cmd = conn.CreateCommand();
        cmd.CommandText = sql;
        using var reader = cmd.ExecuteReader();

        string[] cols = new string[reader.FieldCount];
        for (int i = 0; i < reader.FieldCount; i++)
            cols[i] = reader.GetName(i);

        var rows = new List<string[]>();
        while (reader.Read())
        {
            string[] row = new string[reader.FieldCount];
            for (int i = 0; i < reader.FieldCount; i++)
                row[i] = reader.GetValue(i)?.ToString() ?? "";
            rows.Add(row);
        }
        return (cols, rows);
    }
}

// ===== ReportBuilder.cs =====
using System;
using System.Collections.Generic;

namespace ElectronicsStore
{
    public class ReportBuilder
    {
        private readonly DatabaseManager _db;
        private string _sql = "";
        private string _title = "";
    }
}

```

```

private string[] _headers = Array.Empty<string>();
private int[] _widths = Array.Empty<int>();
private string _footerLabel = "";

public ReportBuilder(DatabaseManager db) { _db = db; }

public ReportBuilder Query(string sql) { _sql = sql; return this; }
public ReportBuilder Title(string title) { _title = title; return this; }
public ReportBuilder Header(params string[] columns) { _headers = columns;
return this; }
    public ReportBuilder ColumnWidths(params int[] widths) { _widths = widths;
return this; }
    public ReportBuilder Footer(string label) { _footerLabel = label; return
this; }

public void Print()
{
    var (columns, rows) = _db.RunSql(_sql);

    if (!string.IsNullOrEmpty(_title))
        Console.WriteLine($"\\n=== {_title} ===");

    string[] displayHeaders = _headers.Length > 0 ? _headers : columns;
    int colCount = displayHeaders.Length;

    int[] widths = _widths.Length >= colCount ? _widths : new
int[colCount];
    if (_widths.Length < colCount)
    {
        for (int i = 0; i < colCount; i++) widths[i] = 25;
    }

    for (int i = 0; i < colCount; i++)
        Console.Write(displayHeaders[i].PadRight(widths[i]));
    Console.WriteLine();

    int totalWidth = 0;
    for (int i = 0; i < colCount; i++) totalWidth += widths[i];
    Console.WriteLine(new string('-', totalWidth));

    foreach (var row in rows)
    {

```



```

        for (int c = 0; c < row.Length && c < colCount; c++)
            Console.Write(row[c].PadRight(widths[c]));
        Console.WriteLine();
    }

    if (!string.IsNullOrEmpty(_footerLabel))
    {
        Console.WriteLine(new string('-', totalWidth));
        Console.WriteLine($"{_footerLabel}: {rows.Count}");
    }
}

}

// ===== dz2.cs =====
using System;
using System.Text;

namespace ElectronicsStore
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.OutputEncoding = Encoding.UTF8;
            Console.InputEncoding = Encoding.UTF8;

            var db = new DatabaseManager();
            db.Init();

            string choice;
            do
            {
                Console.WriteLine("\n=====");
                Console.WriteLine("    МЕНЮ УПРАВЛЕНИЯ МАГАЗИНОМ СМАРТФОНОВ    ");
                Console.WriteLine("=====");
                Console.WriteLine("1 - Вывести справочник производителей");
                Console.WriteLine("2 - Показать полный список смартфонов");
                Console.WriteLine("3 - Внести новый смартфон в базу");
                Console.WriteLine("4 - Модифицировать (изменить) параметры");
                Console.WriteLine("5 - Безвозвратно удалить смартфон");
                Console.WriteLine("6 - Перейти в блок аналитических Отчётов");
            }
            while (choice != "6");
        }
    }
}

```

```

        Console.WriteLine("0 - Завершить работу");
        Console.Write("Введите номер действия: ");
        choice = Console.ReadLine()?.Trim() ?? "";

        switch (choice)
        {
            case "1": ShowBrands(db); break;
            case "2": ShowPhones(db); break;
            case "3": CreatePhone(db); break;
            case "4": ModifyPhone(db); break;
            case "5": RemovePhone(db); break;
            case "6": ViewReports(db); break;
            case "0": Console.WriteLine("\nРабота завершена."); break;
            default: Console.WriteLine("\nОшибка: такого пункта нет.");
        }
        break;
    } while (choice != "0");
}

static void ShowBrands(DatabaseManager db)
{
    Console.WriteLine("\n--- Производители в системе ---");
    foreach (var b in db.GetBrands()) Console.WriteLine($" {b}");
}

static void ShowPhones(DatabaseManager db)
{
    Console.WriteLine("\n--- Модели смартфонов в наличии ---");
    var list = db.GetPhones();
    foreach (var p in list) Console.WriteLine($" {p}");
    Console.WriteLine($"Всего на складе: {list.Count} позиций.");
}

static void CreatePhone(DatabaseManager db)
{
    Console.WriteLine("\n--- Регистрация нового смартфона ---");
    ShowBrands(db);
    Console.Write("Укажите числовой код производителя (ID): ");
    if (!int.TryParse(Console.ReadLine(), out int mfgId)) return;

    Console.Write("Введите название модели: ");
    string name = Console.ReadLine()?.Trim() ?? "";

```

```

        if (string.IsNullOrEmpty(name)) return;

        Console.Write("Укажите розничную цену в рублях: ");
        if (!int.TryParse(Console.ReadLine(), out int price)) return;

        try
        {
            var testPhone = new Smartphone(0, mfgId, name, price);
            db.AddPhone(testPhone.ManufacturerId, testPhone.Name,
testPhone.Price);
            Console.WriteLine("[Успех] Смартфон добавлен.");
        }
        catch (Exception ex)
        {
            Console.WriteLine($"[Ошибка]: {ex.Message}");
        }
    }

    static void ModifyPhone(DatabaseManager db)
    {
        Console.WriteLine("\n--- Изменение параметров смартфона ---");
        Console.Write("Введите ID изменяемого смартфона: ");
        if (!int.TryParse(Console.ReadLine(), out int id)) return;

        var phone = db.GetPhoneById(id);
        if (phone == null)
        {
            Console.WriteLine("[Ошибка] Смартфон не найден.");
            return;
        }

        Console.WriteLine($"Текущие данные: {phone}");
        Console.Write($"Новое наименование [{phone.Name}]: ");
        string nInput = Console.ReadLine()?.Trim() ?? "";
        if (nInput.Length > 0) phone.Name = nInput;

        Console.Write($"Новая цена [{phone.Price}]: ");
        string pInput = Console.ReadLine()?.Trim() ?? "";
        if (pInput.Length > 0 && int.TryParse(pInput, out int newPrice))
        {
            try { phone.Price = newPrice; }

```

```

        catch (Exception ex) { Console.WriteLine($"[Ошибка]:
{ex.Message}"); return; }
    }

    db.UpdatePhone(phone);
    Console.WriteLine("[Успех] Информация обновлена.");
}

static void RemovePhone(DatabaseManager db)
{
    Console.WriteLine("\n--- Удаление позиции ---");
    Console.Write("Введите ID смартфона, который нужно удалить: ");
    if (!int.TryParse(Console.ReadLine(), out int id)) return;

    db.DeletePhone(id);
    Console.WriteLine("[Успех] Запись ликвидирована.");
}

static void ViewReports(DatabaseManager db)
{
    string repChoice;
    do
    {
        Console.WriteLine("\n-----");
        Console.WriteLine("            АНАЛИТИЧЕСКИЕ ОТЧЕТЫ            ");
        Console.WriteLine("-----");
        Console.WriteLine("1 - Прайс-лист смартфонов (JOIN)");
        Console.WriteLine("2 - Подсчет количества моделей (GROUP BY +
COUNT)");
        Console.WriteLine("3 - Расчет средней стоимости (GROUP BY +
AVG)");
        Console.WriteLine("0 - Вернуться в основное меню");
        Console.Write("Выберите отчет: ");
        repChoice = Console.ReadLine()?.Trim() ?? "";

        switch (repChoice)
        {
            case "1":
                new ReportBuilder(db)
                    .Query("SELECT p.phn_name, m.mfg_name, p.phn_price
FROM smartphone p JOIN manufacturer m ON p.mfg_id = m.mfg_id ORDER BY p.phn_name
ASC")

```

```

        .Title("ПОЛНЫЙ ПРАЙС-ЛИСТ СМАРТФОНОВ")
        .Header("Модель смартфона", "Бренд", "Цена (руб.)")
        .ColumnWidths(25, 20, 15)
        .Footer("Всего наименований в прайсе")
        .Print();
        break;
    case "2":
        new ReportBuilder(db)
            .Query("SELECT m.mfg_name, COUNT(*) FROM smartphone p
JOIN manufacturer m ON p.mfg_id = m.mfg_id GROUP BY m.mfg_name")
            .Title("ОБЪЕМ МОДЕЛЬНОГО РЯДА ПО БРЕНДАМ")
            .Header("Бренд производитель", "Количество моделей")
            .ColumnWidths(25, 25)
            .Footer("Проанализировано торговых марок")
            .Print();
        break;
    case "3":
        new ReportBuilder(db)
            .Query("SELECT m.mfg_name, ROUND(AVG(p.phn_price), 2)
FROM smartphone p JOIN manufacturer m ON p.mfg_id = m.mfg_id GROUP BY m.mfg_name")
            .Title("СРЕДНЯЯ ЦЕНА СМАРТФОНОВ ПО БРЕНДАМ")
            .Header("Бренд производитель", "Средняя стоимость")
            .ColumnWidths(25, 25)
            .Footer("Количество исследованных сегментов")
            .Print();
        break;
    }
} while (repChoice != "0");
}
}
}

```

2. Результат работы:

```

=====
      МЕНЮ УПРАВЛЕНИЯ МАГАЗИНОМ СМАРТФОНОВ
=====

```

- 1 - Вывести справочник производителей
- 2 - Показать полный список смартфонов
- 3 - Внести новый смартфон в базу
- 4 - Модифицировать (изменить) параметры смартфона

- 5 - Безвозвратно удалить смартфон
- 6 - Перейти в блок аналитических Отчётов
- 0 - Завершить работу

Введите номер действия: 1

--- Производители в системе ---

- [ID: 1] Apple
- [ID: 2] Samsung
- [ID: 3] Xiaomi
- [ID: 4] Huawei

--- Регистрация нового смартфона ---

--- Производители в системе ---

- [ID: 1] Apple
- [ID: 2] Samsung
- [ID: 3] Xiaomi
- [ID: 4] Huawei

Укажите числовой код производителя (ID): 3

Введите название модели: ка

Укажите розничную цену в рублях: 3

[Успех] Смартфон добавлен.

--- Модели смартфонов в наличии ---

- [ID: 1] iPhone 15 Pro | Код бренда: #1 | Цена: 120000 руб.
- [ID: 2] iPhone 14 | Код бренда: #1 | Цена: 85000 руб.
- [ID: 3] iPhone SE | Код бренда: #1 | Цена: 45000 руб.
- [ID: 4] Galaxy S24 Ultra | Код бренда: #2 | Цена: 130000 руб.
- [ID: 5] Galaxy A55 | Код бренда: #2 | Цена: 38000 руб.
- [ID: 6] Galaxy XCover 7 | Код бренда: #2 | Цена: 42000 руб.
- [ID: 7] Xiaomi 14 Ultra | Код бренда: #3 | Цена: 110000 руб.
- [ID: 8] Redmi Note 13 Pro | Код бренда: #3 | Цена: 28000 руб.
- [ID: 9] POCO F6 | Код бренда: #3 | Цена: 35000 руб.
- [ID: 10] Pura 70 Pro | Код бренда: #4 | Цена: 90000 руб.
- [ID: 11] Nova 12 SE | Код бренда: #4 | Цена: 27000 руб.
- [ID: 12] Mate 60 Pro | Код бренда: #4 | Цена: 105000 руб.
- [ID: 13] ка | Код бренда: #3 | Цена: 3 руб.

Всего на складе: 13 позиций.

Вывод: в ходе выполнения домашнего задания было разработано консольное приложение на языке C# с применением принципов ООП. Реализованы классы-модели Manufactures и Smartfone, класс DatabaseManager для работы с SQLite, класс ReportBuilder с паттерном Fluent Interface. Приложение поддерживает CRUD-операции, импорт данных из CSV и формирование трёх видов отчётов. Дополнительно реализован метод Footer для вывода итоговой строки в отчётах (задание группы 3В).

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. Быков, А. Ю. Решение задач на языках программирования Си и Си++ : методические указания / А. Ю. Быков. — Москва : МГТУ им. Н.Э. Баумана, 2017. — 248 с. — ISBN 978-5-7038-4577-6. — Текст : электронный // Лань : электронно-библиотечная система. — URL: <https://e.lanbook.com/book/103505>
2. Каширин, И. Ю. От Си к Си++ : учебное пособие / И. Ю. Каширин, В. С. Новичков. — 2-е изд., стер. — Москва : Горячая линия-Телеком, 2012. — 334 с. — ISBN 978-5-9912-0259-6. — Текст : электронный // Лань : электронно-библиотечная система. — URL: <https://e.lanbook.com/book/5161>
3. Быков А. Ю. Решение задач на языках программирования Си и Си++ : метод. указания к выполнению лаб. работ / Быков А. Ю. ; МГТУ им. Н. Э. Баумана. - М. : Изд-во МГТУ им. Н. Э. Баумана, 2017. - 244 с. : ил. - ISBN 978-5-7038-4577-6.
4. Иванова Г. С., Ничушкина Т. Н. Объектно-ориентированное программирование : учебник для вузов / Иванова Г. С., Ничушкина Т. Н. ; общ. ред. Иванова Г. С. - М. : Изд-во МГТУ Н.Э.Баумана, 2002. <http://progbook.ru/technologiya-programmirovaniya/582-ivanova-tehnologiya-programmirovaniya.html>
5. Иванова Г. С., Ничушкина Т. Н. Объектно-ориентированное программирование : учебник для вузов / Иванова Г. С., Ничушкина Т. Н. ; общ. ред. Иванова Г. С. - М. : Изд-во МГТУ им. Н. Э. Баумана, 2014. - 455 с. : ил. - Библиогр.: с. 450. - ISBN 978-5-7038-3921-8.
6. Подбельский В.В. Язык Си++: Учебное пособие. – М.: Финансы и статистика, 2003. <http://progbook.ru/c/737-podbelskii-programmiovaniye-na-yazyke-si.html>.
7. Акулов О.А., Медведев Н.В. Информатика. Базовый курс. – М.: Омега-Л, 2006. <http://razym.ru/naukaobraz/obrazov/151874-akulov-oa-medvedev-nv-informatika-bazovyy-kurs.html>

8. Вычислительные методы и программирование. МГУ им. М.В. Ломоносова. ISSN 1726-3522. Журнал входит в 1-й уровень Белого списка научных журналов Минобрнауки России. <https://num-meth.ru/index.php/journal/index>

Дополнительные материалы

1. Иванова Г.С. Технология программирования: Учебник для вузов. – М.: Изд. МГТУ им. Ахо А.В., Хопкрофт Д.Э., Ульман Д.Д. Структуры данных и алгоритмы. – М., Вильямс, 2003. <http://razym.ru/naukaobraz/obrazov/181547-aho-a-ulman-d-hopcroft-d-struktury-dannyh-i-algoritmy.html>
2. Дейтел Х.М., Дейтел П.Дж. Как программировать на C++. – М.: Бином, 2001.
3. Т. Кормен Т., Лейзерсон Ч., Ривест Р. Алгоритмы: построение и анализ. – М. МЦНМО, 2005. <http://rutracker.org/forum/viewtopic.php?t=533181>
4. Джосьютис Н. C++ Стандартная библиотека для профессионалов. – СПб.: Питер, 2004. http://progbook.ru/c/178-dzhosyutis_c_standartnaya_biblioteka.html
5. Подбельский В.В. Стандартный Си++: Учебное пособие. – М.: Финансы и статистика, 2008.
6. Объектно-ориентированное программирование в C++: пер. с англ. / Лафоре Р. - 4-е изд. - СПб.: Питер, 2004. - 923 с. - (Классика computer science). - ISBN 5-94723-302-9.
7. Т. Кормен Т., Лейзерсон Ч., Ривест Р. Алгоритмы: построение и анализ. – М. МЦНМО, 2005.
8. Г. Шилдт. C++. Базовый курс, 3-е издание: Пер с англ. – М.: Издательский дом «Вильямс», 2011. – 624 с.
9. Павловская Т. А. C/C++. Программирование на языке высокого уровня: учебник для вузов / Павловская Т. А. - СПб.: Питер, 2003. - 460 с. - (Учебник для вузов). - ISBN 5-94723-568-4.
10. Бесплатные образовательные программы партнера (VK): <https://education.vk.company/students>